CS445 / SE463 / CS645

Software Requirements Specification and Analysis Spring 2026

Deliverable 4: Functional Model, OCL, and State Diagram

Group Name: The Reservists System
Name: Carrel

Kevin Wang
Chris Wu
Rui Lin

June 23, 2026

1 Project Identification

Item	Description
Group Name	The Reservists
System Name	Carrel
Project Name	Carrel: A Campus Study Space Booking System
Primary Product Perspective	A new campus-wide study space booking system, not a successor to a specific existing system.

Carrel is a campus study space booking system designed to help students find, reserve, check in to, and cancel study space bookings. The system also supports administrators who manage room inventory, booking rules, maintenance closures, no-show policies, disputes, and administrative overrides. The main goal is to make study space allocation easier, fairer, and less wasteful by reducing ghost reservations and centralizing booking information.

2 Updated Domain Model

The updated model is intended to support the functional model, OCL restrictions, and state diagram while preserving the previously defined objects: User, Student, Administrator, Campus, Building, Floor, StudySpace, IndividualCarrel, GroupStudyRoom, OpenStudyArea, Amenity, RoomQRCode, Booking, CheckInCode, CheckIn, Cancellation, CalendarInvite, NoShowStrike, Dispute, BookingPolicy, BookingRule, MaintenanceClosure, AdministrativeOverride, UniversityIdentityService, and CalendarService. To support lifecycle behavior, enforcement logic, and policy constraints, this update adds specific variables, including Booking.checkInDeadline, Booking.releasedAt, Booking.cancelledAt, CheckInCode.isUsed, NoShowStrike.isActive, Dispute.reason, BookingPolicy limit values, MaintenanceClosure.status, Student.activeBookingLimit, and Student.isSuspended.

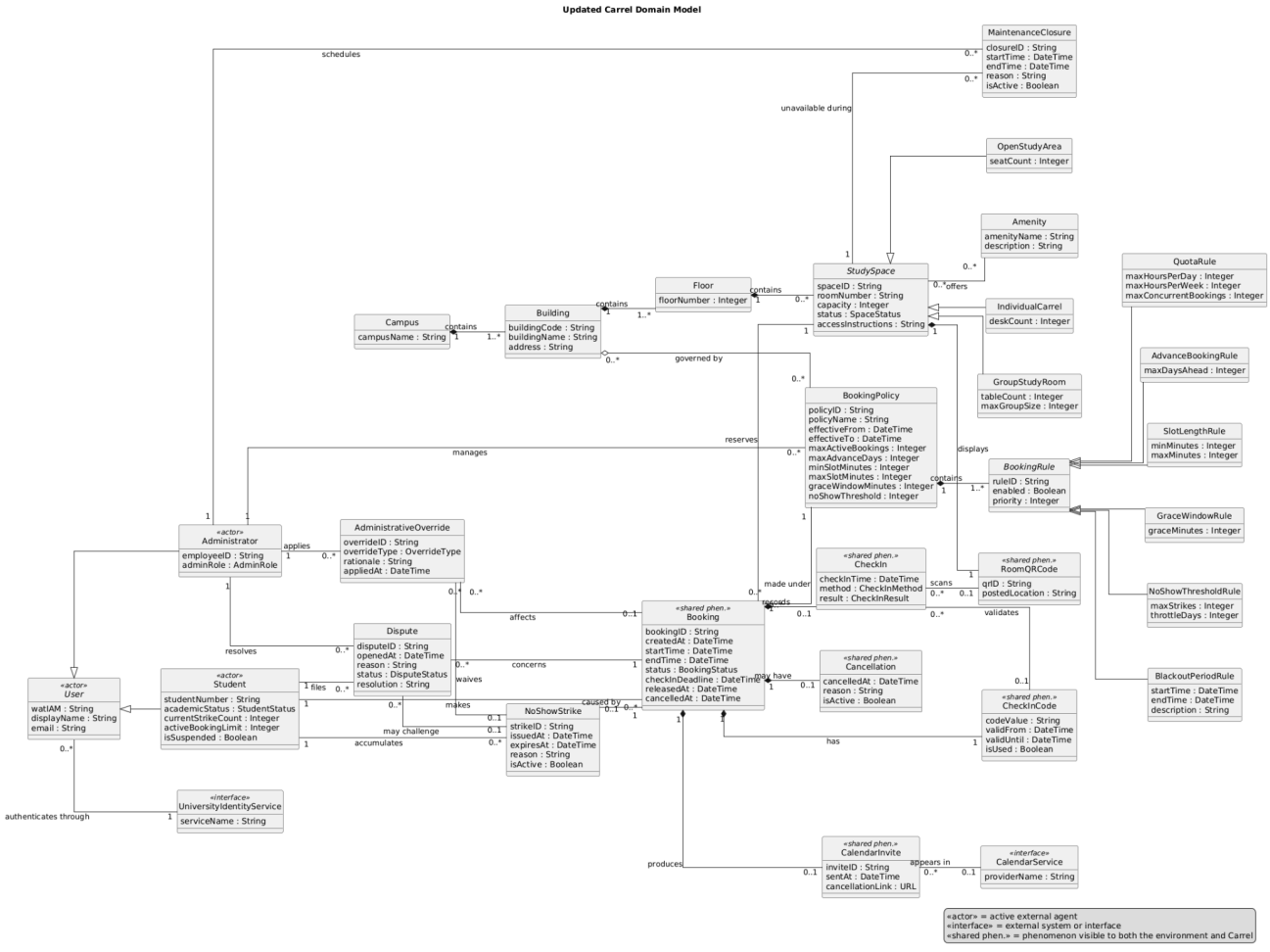


Figure 1: Updated Carrel Domain Model

3 Updated Use Case Diagram

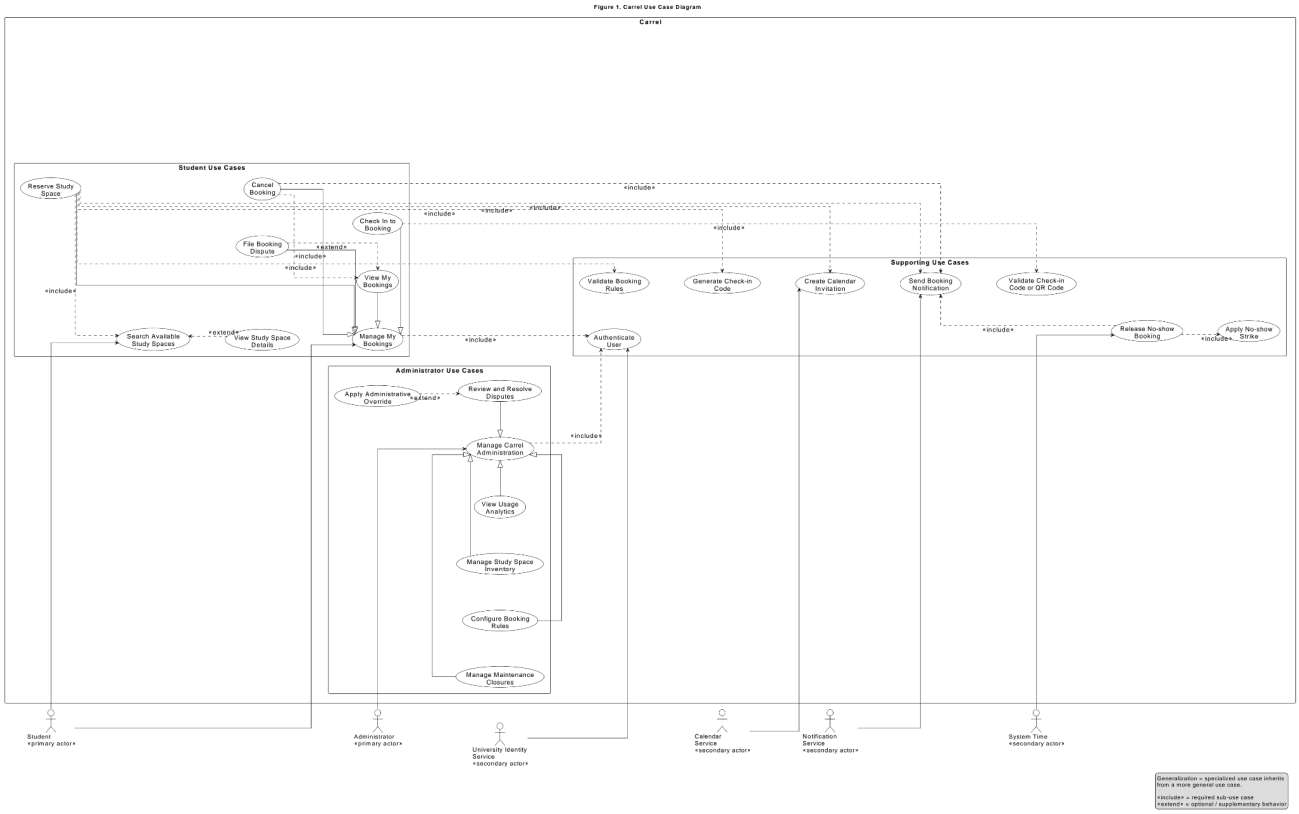


Figure 2: Updated Carrel Use Case Diagram

Brief Use Case Descriptions

UC1 Search Available Study Spaces: A student searches for bookable spaces by date, time, building, capacity, and amenities.

UC2 View Study Space Details: A student views location, capacity, amenities, access notes, and availability for a selected space.

UC3 Reserve Study Space: A student reserves an available study space for a selected time slot, subject to booking rules.

UC4 View My Bookings: A student views current, upcoming, and relevant past bookings.

UC5 Check In to Booking: A student confirms physical presence for an active booking during the check-in window.

UC6 Cancel Booking: A student cancels a future booking so the space returns to the available pool.

UC7 File Booking Dispute: A student submits a dispute about a booking, no-show record, or room conflict.

UC8 Manage Study Space Inventory: An administrator adds, edits, retires, or updates bookable study spaces.

UC9 Configure Booking Rules: An administrator configures quotas, slot lengths, advance windows, grace windows, and no-show rules.

UC10 Manage Maintenance Closures: An administrator blocks spaces for maintenance, repairs, cleaning, or temporary closures.

UC11 Review and Resolve Disputes: An administrator reviews submitted disputes and records a resolution.

UC12 Apply Administrative Override: An administrator overrides a booking, cancellation, dispute result, or enforcement outcome when justified.

UC13 View Usage Analytics: An administrator views usage patterns, no-show rates, peak demand, and under-used spaces.

UC14 Authenticate User: Carrel verifies a user identity and role using the University Identity Service.

UC15 Send Notification: Carrel sends confirmations, cancellations, reminders, no-show notices, and dispute updates.

UC16 Create Calendar Invitation: Carrel creates a calendar event after a booking is confirmed.

UC17 Release No-show Booking: System Time triggers release of an unclaimed booking after the grace window.

4 Functional Model

The following five functions use the required notation: pre, modifies, post, and exception. Functions 1 and 5 are complex functions because they involve multiple domain objects, policy checks, lifecycle state changes, and external service interactions.

Function 1: reserveStudySpace(student, studySpace, startTime, endTime): Booking
signature:

reserveStudySpace(s: Student, sp: StudySpace, t1: DateTime, t2: DateTime) -> b: Booking

pre:

s ∈ Student ∧ sp ∈ StudySpace
s.isActive = true
s.isSuspended = false
sp.isBookable = true
sp.status = Available
t1 < t2
activeBookings(s) =
{ b ∈ Booking | b.student = s ∧ b.status ∈ {Confirmed, AwaitingCheckIn, CheckedIn}
∧ b.endTime > now() }
|activeBookings(s)| < s.activeBookingLimit
∀ b ∈ Booking:
b.studySpace = sp ∧ b.status ∈ {Confirmed, AwaitingCheckIn, CheckedIn}
=> (t2 <= b.startTime ∨ t1 >= b.endTime)
∀ c ∈ MaintenanceClosure:
c.studySpace = sp ∧ c.status = Active
=> (t2 <= c.startTime ∨ t1 >= c.endTime)
∀ r ∈ applicableRules(s, sp, t1, t2): r.isSatisfied = true

modifies:

Booking
CheckInCode
CalendarInvite
Notification
CalendarService.calendarSyncStatus
s.bookings
sp.bookings

post:

∃ b ∈ Booking:
b.student = s
∧ b.studySpace = sp
∧ b.startTime = t1
∧ b.endTime = t2
∧ b.status = Confirmed
∧ b.checkInDeadline = t1 + policy(sp).checkInGracePeriod
∧ b ∈ s.bookings
∧ b ∈ sp.bookings
∃ code ∈ CheckInCode:
code.booking = b
∧ code.isUsed = false

∧ code.expiresAt = b.checkInDeadline
∃ invite ∈ CalendarInvite:
invite.booking = b
∧ invite.student = s
∧ invite.startTime = t1
∧ invite.endTime = t2
∃ n ∈ Notification:
n.recipient = s
∧ n.type = BookingConfirmation
∧ n.subject = b

exception:

s ∉ Student ∨ sp ∉ StudySpace
=> result = InvalidInput ∧ Booking' = Booking
s.isActive = false ∨ s.isSuspended = true
=> result = StudentNotEligible ∧ Booking' = Booking
sp.isBookable = false ∨ sp.status ≠ Available
=> result = StudySpaceUnavailable ∧ Booking' = Booking
t1 >= t2
=> result = InvalidTimeRange ∧ Booking' = Booking
∃ b ∈ Booking:
b.studySpace = sp ∧ b.status ∈ {Confirmed, AwaitingCheckIn, CheckedIn}
∧ overlaps([t1, t2), [b.startTime, b.endTime))
=> result = BookingConflict ∧ Booking' = Booking
∃ c ∈ MaintenanceClosure:
c.studySpace = sp ∧ c.status = Active
∧ overlaps([t1, t2), [c.startTime, c.endTime))
=> result = MaintenanceConflict ∧ Booking' = Booking
|activeBookings(s)| >= s.activeBookingLimit
=> result = ActiveBookingLimitExceeded ∧ Booking' = Booking
∃ r ∈ applicableRules(s, sp, t1, t2): r.isSatisfied = false
=> result = RuleViolation(r) ∧ Booking' = Booking

Function 2: checkInToBooking(student, booking, checkInCode, currentTime): CheckIn

signature:

checkInToBooking(s: Student, b: Booking, code: CheckInCode, t: DateTime) -> ci: CheckIn

pre:

s ∈ Student ∧ b ∈ Booking ∧ code ∈ CheckInCode
b.student = s
b.status ∈ {Confirmed, AwaitingCheckIn}
code.booking = b

code.isUsed = false
t <= b.checkInDeadline
b.checkIn = null

modifies:

CheckIn
b.status
b.checkIn
code.isUsed

post:

∃ ci ∈ CheckIn:
ci.booking = b
∧ ci.student = s
∧ ci.checkInTime = t
b.checkIn = ci
b.status = CheckedIn
code.isUsed = true

exception:

b.student ≠ s
=> result = Unauthorized ∧ CheckIn' = CheckIn
b.status ∉ {Confirmed, AwaitingCheckIn}
=> result = InvalidBookingState ∧ CheckIn' = CheckIn
code.booking ≠ b ∨ code.isUsed = true
=> result = InvalidCheckInCode ∧ CheckIn' = CheckIn
t > b.checkInDeadline
=> result = CheckInWindowExpired ∧ CheckIn' = CheckIn
b.checkIn ≠ null
=> result = AlreadyCheckedIn ∧ CheckIn' = CheckIn

Function 3: cancelBooking(student, booking, currentTime, reason): Cancellation

signature:

cancelBooking(s: Student, b: Booking, t: DateTime, reason: String) -> c: Cancellation

pre:

s ∈ Student ∧ b ∈ Booking
b.student = s
b.status ∈ {Confirmed, AwaitingCheckIn}
t < b.startTime ∨ t <= b.checkInDeadline

modifies:

Cancellation
b.status
b.cancelledAt

Notification
post:
∃ c ∈ Cancellation:
c.booking = b
∧ c.student = s
∧ c.cancelledAt = t
∧ c.reason = reason
b.status = Cancelled
b.cancelledAt = t
∃ n ∈ Notification:
n.recipient = s
∧ n.type = BookingCancellation
∧ n.subject = b
exception:
b ∉ Booking
=> result = BookingNotFound ∧ Cancellation' = Cancellation
b.student ≠ s
=> result = Unauthorized ∧ Cancellation' = Cancellation
b.status = Cancelled
=> result = AlreadyCancelled ∧ Cancellation' = Cancellation
b.status ∈ {CheckedIn, Completed, NoShowReleased}
=> result = CancellationNotAllowed ∧ Cancellation' = Cancellation
t > b.checkInDeadline ∧ t >= b.startTime
=> result = CancellationWindowClosed ∧ Cancellation' = Cancellation

Function 4: fileBookingDispute(student, booking, reason, submittedAt): Dispute
signature:
fileBookingDispute(s: Student, b: Booking, reason: String, t: DateTime) -> d: Dispute
pre:
s ∈ Student ∧ b ∈ Booking
b.student = s
reason ≠ null ∧ trim(reason) ≠ ""
¬∃ d ∈ Dispute:
d.booking = b ∧ d.student = s ∧ d.status ∈ {Submitted, UnderReview}
modifies:
Dispute
b.disputes
Notification
post:
∃ d ∈ Dispute:

d.booking = b
∧ d.student = s
∧ d.reason = reason
∧ d.status = Submitted
∧ d.submittedAt = t
∧ d ∈ b.disputes
∃ n ∈ Notification:
n.recipient = s
∧ n.type = DisputeSubmitted
∧ n.subject = d

exception:

b ∉ Booking
=> result = BookingNotFound ∧ Dispute' = Dispute

b.student ≠ s
=> result = Unauthorized ∧ Dispute' = Dispute

reason = null ∨ trim(reason) = ""
=> result = EmptyDisputeReason ∧ Dispute' = Dispute

∃ d ∈ Dispute:
d.booking = b ∧ d.student = s ∧ d.status ∈ {Submitted, UnderReview}
=> result = DuplicateOpenDispute ∧ Dispute' = Dispute

Function 5: releaseNoShowBooking(booking, currentTime): NoShowStrike

signature:

releaseNoShowBooking(b: Booking, t: DateTime) -> strike: NoShowStrike

pre:

b ∈ Booking
b.status ∈ {Confirmed, AwaitingCheckIn}
b.checkIn = null
t > b.checkInDeadline
¬∃ strike ∈ NoShowStrike: strike.booking = b

modifies:

NoShowStrike
b.status
b.releasedAt
b.noShowStrike
b.student.currentStrikeCount
Notification

post:

b.status = NoShowReleased
b.releasedAt = t

∃ strike ∈ NoShowStrike:
strike.booking = b
∧ strike.student = b.student
∧ strike.issuedAt = t
∧ strike.isActive = true
∧ b.noShowStrike = strike
b.student.currentStrikeCount =
b.student.currentStrikeCount@pre + 1
availableAfterRelease(b.studySpace, b.endTime) = true
∃ n ∈ Notification:
n.recipient = b.student
∧ n.type = NoShowReleased
∧ n.subject = b

exception:

b ∉ Booking
=> result = BookingNotFound ∧ NoShowStrike' = NoShowStrike
b.status ∉ {Confirmed, AwaitingCheckIn}
=> result = InvalidBookingState ∧ NoShowStrike' = NoShowStrike
b.checkIn ≠ null
=> result = AlreadyCheckedIn ∧ NoShowStrike' = NoShowStrike
t <= b.checkInDeadline
=> result = CheckInWindowStillOpen ∧ NoShowStrike' = NoShowStrike
∃ strike ∈ NoShowStrike: strike.booking = b
=> result = NoShowAlreadyReleased ∧ NoShowStrike' = NoShowStrike

5 OCL Restrictions

OCL 1: Student active booking limit
Natural language: A student may not have more active or upcoming bookings than their active booking limit.

```
context Student inv ActiveBookingLimit:
  self.bookings->select(b |
    b.status = 'Confirmed' or
    b.status = 'AwaitingCheckIn' or
    b.status = 'CheckedIn'
  )->size() <= self.activeBookingLimit
```

OCL 2: Valid booking time range
Natural language: Every booking must start before it ends.

```
context Booking inv ValidBookingTimeRange:
  self.startTime < self.endTime
```

OCL 3: Booking must have one student and one study space
Natural language: Each booking must be linked to exactly one student and one study space.

```
context Booking inv BookingHasRequiredParties:
  not self.student.oclIsUndefined() and
  not self.studySpace.oclIsUndefined()
```

OCL 4: Study space capacity must be positive
Natural language: Every study space must have a positive capacity.

```
context StudySpace inv PositiveCapacity:
  self.capacity > 0
```

OCL 5: Individual carrels have capacity one
Natural language: If a study space is an IndividualCarrel, its capacity must be exactly one.

```
context StudySpace inv IndividualCarrelCapacity:
  self.oclIsTypeOf(IndividualCarrel) implies
  self.capacity = 1
```

OCL 6: Group rooms have capacity greater than one
Natural language: If a study space is a GroupStudyRoom, it must support more than one person.

```
context StudySpace inv GroupStudyRoomCapacity:
  self.oclIsTypeOf(GroupStudyRoom) implies
  self.capacity > 1
```

OCL 7: Cancellation must occur no later than the booking start time
Natural language: A cancellation record for a booking must be created no later than the booking start time.

```
context Cancellation inv CancellationBeforeStart:
  self.cancelledAt <= self.booking.startTime
```

OCL 8: No-show release requires a missed check-in
Natural language: A booking can be marked as no-show released only if it has no check-in and its check-in deadline has passed.

```
context Booking inv NoShowRequiresMissedCheckIn:
  self.status = 'NoShowReleased' implies
  self.checkIn.oclIsUndefined() and
  self.checkInDeadline < self.releasedAt
```

OCL 9: Dispute must refer to a booking or no-show strike
Natural language: Each dispute must be connected to a booking or to a no-show strike.

```
context Dispute inv DisputeHasSubject:
  not self.booking.oclIsUndefined() or
  not self.noShowStrike.oclIsUndefined()
```

OCL 10: Maintenance closures cannot overlap active bookings
Natural language: A maintenance closure for a study space must not overlap any active booking for that same study space.

```
context MaintenanceClosure inv NoActiveBookingDuringClosure:
  self.studySpace.bookings->select(b |
    b.status = 'Confirmed' or
    b.status = 'AwaitingCheckIn' or
    b.status = 'CheckedIn'
  )->forAll(b |
    b.endTime <= self.startTime or
    b.startTime >= self.endTime
  )
```

OCL 11: Check-in code belongs to an active booking
Natural language: A check-in code may not be used for a cancelled or no-show released booking.

```
context CheckInCode inv CodeBelongsToActiveBooking:
  self.booking.status <> 'Cancelled' and
  self.booking.status <> 'NoShowReleased'
```

OCL 12: Booking policy values are valid
Natural language: Booking policy limit values must be positive and internally consistent.

```
context BookingPolicy inv ValidPolicyValues:
  self.maxActiveBookings > 0 and
  self.maxAdvanceDays > 0 and
  self.minSlotMinutes > 0 and
  self.maxSlotMinutes >= self.minSlotMinutes and
  self.graceWindowMinutes >= 0 and
  self.noShowThreshold >= 0
```

6 State Diagram

The state diagram models Carrel's generic user-interface states. It separates student-facing booking activity from administrator-facing management activity while keeping both within the authenticated Carrel session.

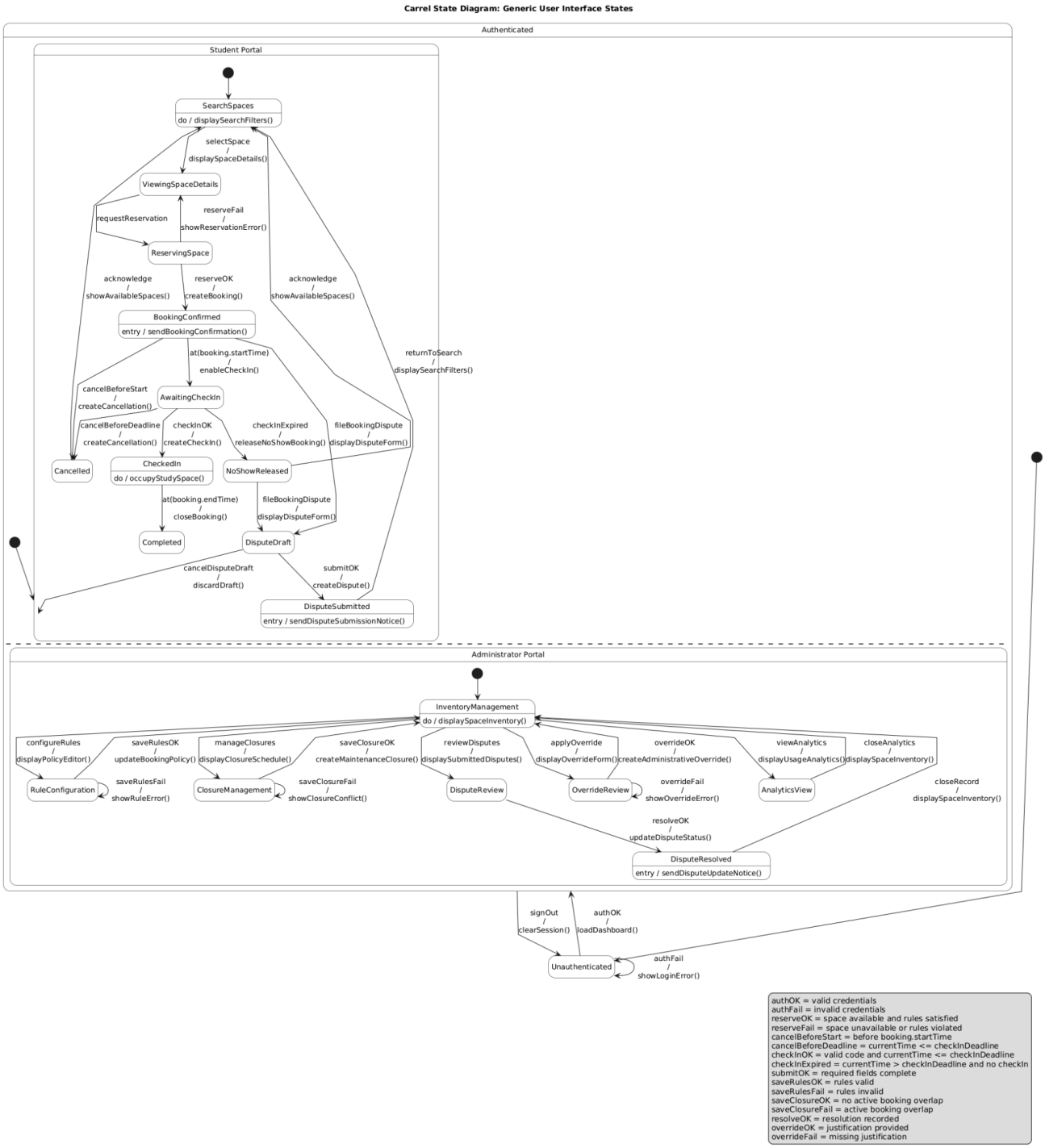


Figure 3: Carrel State Diagram